

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

TUGAS BESAR

Milestone 4

(Kelompok TBFO)



Dipersiapkan oleh:

13524044	Narendra Dharma Wistara M
13524060	Steven Tan
13524090	Nashiruddin Akram
13524100	Arghawisesa Dwinanda Arham

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

Daftar Isi

Daftar Isi.....	1
Teori Singkat.....	2
A. Intermediate Code.....	3
B. Decorated AST.....	3
C. Intermediate Code Generator.....	4
D. Instruction Set.....	5
E. Translasi Control Flow.....	6
F. Interpreter.....	7
G. Stack Machine.....	7
H. Konvensi Stack Machine dan Instruksi Intermediate Code.....	8
I. Validasi Runtime.....	10
J. Error Handling pada Interpreter.....	10
Perancangan & Implementasi.....	12
A. Teknologi yang Digunakan.....	12
B. Gambaran Umum Arsitektur.....	12
C. Struktur Program.....	13
D. Fungsi/Kelas Utama.....	15
1. Komponen Intermediate Code.....	16
2. Komponen Code Generator.....	17
3. Komponen Interpreter dan Runtime.....	19
E. Alur Kerja Program.....	21
F. Implementasi Pembacaan dan Validasi Decorated AST.....	22
G. Implementasi Normalisasi Layout Runtime.....	24
H. Implementasi Representasi Intermediate Code.....	25
I. Implementasi Intermediate Code Generator.....	26
1. Pembangkitan Program dan Subprogram.....	27
2. Pembangkitan Literal, Identifier, dan Ekspresi.....	27
3. Pembangkitan Assignment dan L-Value.....	28
4. Pembangkitan Control Flow.....	28
J. Implementasi Runtime Value dan Memori Eksekusi.....	29
K. Implementasi Interpreter.....	29
L. Implementasi Procedure, Function, Nested Scope, dan Rekursi.....	30
M. Implementasi Array, Record, dan Composite Value.....	31
N. Implementasi Input dan Output.....	32
O. Implementasi Error Handling dan Runtime Safety.....	33
1. Stack Overflow dan Underflow.....	34
2. Perlindungan Frame dan Memori.....	34
3. Validasi Call dan Return.....	34
4. Bounds Checking.....	34

5. Validasi Operasi Numerik.....	34
6. Validasi Tipe Runtime.....	34
P. Implementasi Output Akhir dan Justifikasi Rancangan.....	35
Pengujian.....	37
Pengujian 1.....	37
Pengujian 2.....	37
Pengujian 3.....	38
Pengujian 4.....	41
Pengujian 5.....	41
Pengujian 6.....	42
Pengujian 7.....	45
Pengujian 8.....	45
Kesimpulan dan Saran.....	47
Kesimpulan.....	47
Saran.....	48
Lampiran.....	49
Referensi.....	50

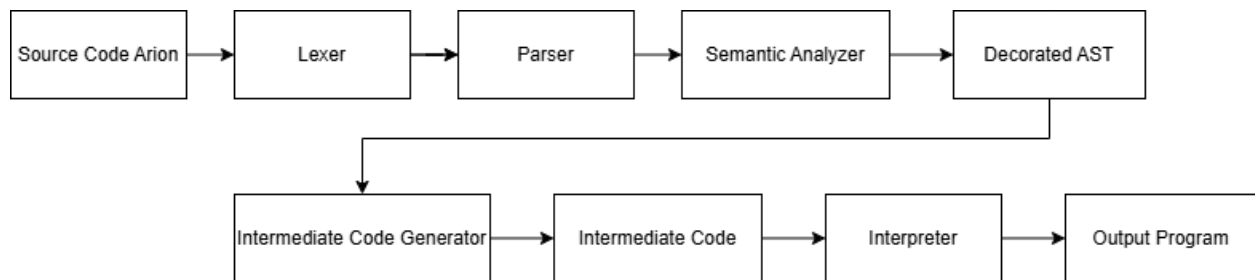
Teori Singkat

A. Intermediate Code

Intermediate Code adalah representasi perantara dari program yang berada di antara hasil analisis semantik dan proses eksekusi. Pada milestone sebelumnya, source code Arion telah melalui proses lexical analysis, syntax analysis, dan semantic analysis hingga menghasilkan struktur program yang sudah valid. Struktur tersebut kemudian perlu diubah menjadi bentuk instruksi yang lebih sederhana agar dapat dijalankan oleh interpreter.

Pada Milestone 4, input utama yang digunakan adalah **Decorated AST**, sedangkan output yang dihasilkan adalah **Intermediate Code** dan output program sebenarnya apabila terdapat perintah seperti `write` atau `writeln`. Intermediate Code digunakan agar program tidak perlu dieksekusi langsung dari struktur pohon, melainkan dapat dijalankan sebagai daftar instruksi linear yang lebih mudah diproses oleh mesin eksekusi.

Secara umum, alur pada milestone ini dapat digambarkan sebagai berikut.



Dengan adanya Intermediate Code, struktur program yang awalnya berbentuk pohon dapat diratakan menjadi instruksi-instruksi sederhana. Setiap instruksi akan merepresentasikan satu operasi tertentu, seperti memuat nilai, menyimpan nilai, melakukan operasi aritmatika, melakukan percabangan, atau mencetak output.

B. Decorated AST

Decorated AST adalah Abstract Syntax Tree yang sudah dilengkapi dengan informasi semantik. Jika AST biasa hanya merepresentasikan struktur program, maka Decorated AST juga menyimpan informasi tambahan seperti tipe data, scope, dan referensi terhadap tabel simbol. Informasi ini dibutuhkan agar proses pembangkitan Intermediate Code dapat dilakukan dengan benar.

Sebagai contoh, pada ekspresi:

$$x := a + b$$

Intermediate Code Generator perlu mengetahui apakah a dan b bertipe integer, real, atau tipe lainnya. Informasi tersebut tidak cukup hanya didapat dari struktur sintaks, tetapi harus berasal dari hasil semantic analysis. Oleh karena itu, Decorated AST menjadi sumber utama dalam proses pembentukan Intermediate Code.

Secara sederhana, Decorated AST dapat dipahami sebagai struktur program yang tidak hanya menyimpan bentuk program, tetapi juga makna dari setiap node. Dengan demikian, setiap node pada Decorated AST dapat diterjemahkan menjadi instruksi yang sesuai.

C. Intermediate Code Generator

Intermediate Code Generator adalah komponen yang bertugas mengubah Decorated AST menjadi daftar instruksi Intermediate Code. Pada Milestone 4, proses pembangkitan Intermediate Code dilakukan menggunakan pendekatan berbasis Depth First Search. Artinya, generator akan menelusuri node-node pada Decorated AST dari bagian terdalam terlebih dahulu, kemudian menghasilkan instruksi sesuai jenis node yang sedang diproses.

Misalnya, untuk assignment berikut:

$$x := 10$$

generator perlu menghasilkan instruksi untuk memuat literal 10, kemudian menyimpannya ke alamat variabel x .

```
LIT 0 10
STO 0 3
```

Contoh lain, untuk ekspresi:

$$y := x * 2$$

generator perlu memuat nilai x , memuat literal 2, melakukan operasi perkalian, lalu menyimpan hasilnya ke variabel y .

```
LOD 0 3
```

```
LIT 0 2
```

```
OPR 0 4
```

```
STO 0 4
```

Dengan mekanisme ini, ekspresi yang semula berbentuk pohon dapat diterjemahkan menjadi instruksi linear yang dapat dieksekusi satu per satu oleh interpreter.

D. Instruction Set

Instruction set adalah kumpulan instruksi yang digunakan dalam Intermediate Code. Setiap instruksi memiliki fungsi tertentu dalam proses eksekusi program. Pada Arion, instruksi utama yang digunakan antara lain `LIT`, `LOD`, `STO`, `CAL`, `INT`, `JMP`, `JPC`, `OPR`, dan `RET`.

Beberapa instruksi utama dapat dijelaskan sebagai berikut.

```
LIT v : memasukkan nilai literal v ke stack
```

```
LOD a : memuat nilai dari address a
```

```
STO a : menyimpan nilai ke address a
```

```
CAL l : memanggil fungsi atau prosedur pada line l
```

```
INT m : menginisialisasi memori sebesar m
```

```
JMP l : lompat tanpa syarat ke line l
```

```
JPC l : lompat ke line l jika kondisi tidak terpenuhi
```

```
OPR o : menjalankan operasi tertentu
```

```
RET : keluar dari fungsi atau prosedur
```

Instruksi `OPR` digunakan untuk menjalankan operasi khusus, seperti operasi aritmatika, operasi perbandingan, dan operasi output. Contohnya, `OPR 0 2` digunakan untuk operasi penjumlahan, `OPR 0 4` untuk perkalian, `OPR 0 11` untuk perbandingan lebih besar dari, dan `OPR 0 14` untuk `writeln`.

Dengan adanya instruction set ini, interpreter tidak perlu memahami langsung sintaks bahasa Arion. Interpreter cukup membaca instruksi yang sudah dibentuk oleh Intermediate Code Generator.

E. Translasi Control Flow

Control flow adalah mekanisme yang mengatur alur eksekusi program, seperti percabangan dan perulangan. Pada bahasa tingkat tinggi, control flow biasanya ditulis dalam bentuk `if-else` atau `while`. Namun, pada Intermediate Code, struktur tersebut perlu diubah menjadi instruksi linear menggunakan mekanisme `jump`.

Pada statement `if-else`, kondisi akan dievaluasi terlebih dahulu. Jika kondisi tidak terpenuhi, eksekusi akan melompat ke blok `else` menggunakan instruksi `JPC`. Jika kondisi terpenuhi, blok `then` dijalankan, lalu eksekusi melompat ke akhir blok `if-else` menggunakan `JMP`.

Contoh struktur `if-else`:

```
if x > 5 then
    y := x * 2
else
    y := x div 2
```

dapat diterjemahkan menjadi bentuk Intermediate Code seperti berikut.

```
LOD 0 3
LIT 0 5
OPR 0 11
JPC 0 12
LOD 0 3
LIT 0 2
OPR 0 4
STO 0 4
```

```
JMP 0 16  
LOD 0 3  
LIT 0 2  
OPR 0 5  
STO 0 4
```

Pada statement `while`, interpreter perlu mengevaluasi kondisi di awal loop. Jika kondisi tidak terpenuhi, eksekusi keluar dari loop. Jika kondisi terpenuhi, body loop dijalankan, kemudian eksekusi melompat kembali ke bagian pengecekan kondisi.

Dengan menggunakan `JMP` dan `JPC`, struktur program yang awalnya bercabang atau berulang dapat tetap direpresentasikan sebagai daftar instruksi linear.

F. Interpreter

Interpreter adalah komponen yang bertugas menjalankan Intermediate Code. Interpreter bekerja seperti virtual machine sederhana yang membaca instruksi satu per satu, mengelola memori, menjalankan operasi, mengatur alur eksekusi, dan menghasilkan output program.

Secara umum, interpreter bekerja melalui tiga tahap utama, yaitu:

1. **Fetch**, yaitu mengambil instruksi yang sedang ditunjuk oleh Instruction Pointer.
2. **Decode**, yaitu membaca jenis instruksi dan operand yang digunakan.
3. **Execute**, yaitu menjalankan instruksi tersebut.

Instruction Pointer atau Program Counter digunakan untuk menyimpan posisi instruksi yang sedang dijalankan. Secara default, Instruction Pointer akan bertambah satu setelah sebuah instruksi selesai dieksekusi. Namun, jika instruksi yang dijalankan adalah `JMP` atau `JPC`, nilai Instruction Pointer dapat berubah ke alamat instruksi lain.

Dengan mekanisme ini, interpreter dapat menjalankan program secara berurutan, sekaligus tetap mendukung percabangan dan perulangan.

G. Stack Machine

Pada Milestone 4, interpreter Arion menggunakan konsep Stack Machine. Stack Machine adalah model eksekusi yang menggunakan stack sebagai tempat utama untuk menyimpan

nilai sementara, operand operasi, hasil perhitungan, dan informasi eksekusi lainnya. Stack bekerja dengan prinsip Last In, First Out, yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan.

Sebagai contoh, pada ekspresi:

$$x + 5$$

interpreter akan memuat nilai x ke stack, kemudian memuat literal 5 ke stack. Setelah itu, instruksi operasi penjumlahan akan mengambil dua nilai teratas dari stack, menghitung hasilnya, lalu mengembalikan hasil tersebut ke stack.

Alur sederhananya adalah sebagai berikut.

LOD 0 3	→ push nilai x
LIT 0 5	→ push nilai 5
OPR 0 2	→ pop dua nilai, jumlahkan, lalu push hasil

Stack juga digunakan untuk menyimpan informasi memori program, seperti static link, dynamic link, return address, variabel lokal, serta nilai sementara. Oleh karena itu, pengelolaan stack menjadi bagian penting dalam eksekusi program Arion.

H. Konvensi Stack Machine dan Instruksi Intermediate Code

Pada Milestone 4, interpreter Arion menggunakan model eksekusi **Stack Machine**. Stack Machine adalah model eksekusi yang memakai stack untuk menyimpan operand, nilai sementara, hasil operasi, dan informasi eksekusi program. Stack bekerja dengan prinsip **Last In, First Out** atau LIFO, yaitu nilai yang terakhir dimasukkan akan menjadi nilai pertama yang diambil kembali.

Dalam model ini, instruksi Intermediate Code bekerja dengan mengubah isi stack. Sebagai contoh, ekspresi:

$$y := (3 + 4) * x$$

dapat diterjemahkan menjadi urutan instruksi berikut.

```
LIT 3  
LIT 4  
ADD  
LOD x  
MUL  
STO y
```

Instruksi `LIT` digunakan untuk memasukkan literal ke stack, `LOD` digunakan untuk memuat nilai dari memori ke stack, sedangkan `STO` digunakan untuk mengambil nilai teratas stack dan menyimpannya ke alamat tertentu. Operasi seperti `ADD`, `SUB`, `MUL`, dan `DIV` mengambil operand dari stack, menghitung hasilnya, lalu memasukkan hasil tersebut kembali ke stack.

Secara sederhana, perubahan stack pada contoh tersebut adalah:

```
LIT 3    -> [3]  
LIT 4    -> [3, 4]  
ADD      -> [7]  
LOD x    -> [7, nilai_x]  
MUL      -> [7 * nilai_x]  
STO y    -> []
```

Dalam pengelolaan stack, terdapat dua pointer penting, yaitu **SP** dan **BP**. **SP (Stack Pointer)** menunjuk elemen teratas stack dan berubah ketika terjadi operasi `push` atau `pop`. Sementara itu, **BP (Base Pointer)** menunjuk awal frame fungsi atau prosedur, sehingga variabel lokal dapat diakses relatif terhadap frame yang sedang aktif. Guidebook Milestone 4 juga menjelaskan bahwa Intermediate Code Arion dirancang berbasis Stack Machine dengan konvensi instruksi seperti `LIT`, `LOD`, `STO`, `JMP`, `JPC`, `CAL`, dan `RET`.

I. Validasi Runtime

Selain menjalankan Intermediate Code, interpreter juga perlu melakukan **validasi runtime**. Validasi runtime adalah proses pemeriksaan saat program dieksekusi untuk memastikan instruksi tidak menyebabkan kondisi yang tidak valid, seperti kesalahan tipe data, akses memori yang salah, atau penggunaan stack yang tidak sesuai.

Beberapa kondisi error yang perlu diperhatikan adalah **type mismatch**, **stack overflow**, **stack underflow**, **invalid jump**, dan **out-of-bounds access**. Type mismatch terjadi ketika nilai yang diproses tidak sesuai dengan tipe yang diharapkan. Kesalahan ini dapat dicek menggunakan informasi tipe yang terdapat pada Decorated AST.

Stack overflow terjadi ketika stack melewati batas kapasitas memori, sedangkan stack underflow terjadi ketika interpreter mencoba mengambil nilai dari stack yang kosong. Invalid jump terjadi ketika instruksi `JMP` atau `JPC` mengarah ke alamat instruksi yang tidak valid. Sementara itu, out-of-bounds access terjadi ketika instruksi `LOD` atau `STO` mencoba mengakses alamat memori di luar batas yang diperbolehkan.

Secara ringkas, validasi runtime dapat dirangkum sebagai berikut.

Type Mismatch → cek tipe berdasarkan Decorated AST

Stack Overflow → cek SP terhadap batas atas stack

Stack Underflow → cek saat instruksi melakukan pop dari stack kosong

Invalid Jump → cek validitas target `JMP/JPC`

OOB Access → cek alamat `LOD/STO` terhadap batas memori

Dengan validasi runtime, interpreter dapat menghentikan eksekusi secara aman ketika terjadi kesalahan. Hal ini membuat interpreter lebih stabil, mudah diuji, dan tidak langsung crash ketika menjalankan Intermediate Code yang bermasalah.

J. Error Handling pada Interpreter

Selain menjalankan program yang benar, interpreter juga perlu mampu menangani kesalahan yang mungkin terjadi saat eksekusi. Error handling diperlukan agar program tidak langsung crash ketika menemukan kondisi yang tidak valid.

Beberapa contoh kesalahan yang dapat terjadi pada interpreter adalah stack overflow, stack underflow, akses memori di luar batas, target jump yang tidak valid, serta overflow atau underflow pada operasi numerik. Stack overflow terjadi ketika terlalu banyak data atau stack frame dimasukkan ke stack. Stack underflow terjadi ketika interpreter mencoba mengambil nilai dari stack yang kosong. Invalid jump target terjadi ketika instruksi `JMP` atau `JPC` mengarah ke baris instruksi yang tidak valid.

Dengan adanya error handling, interpreter dapat menghentikan eksekusi secara aman dan menampilkan pesan error yang jelas. Hal ini penting agar program lebih mudah diuji, diperbaiki, dan dikembangkan pada tahap berikutnya.

Perancangan & Implementasi

A. Teknologi yang Digunakan

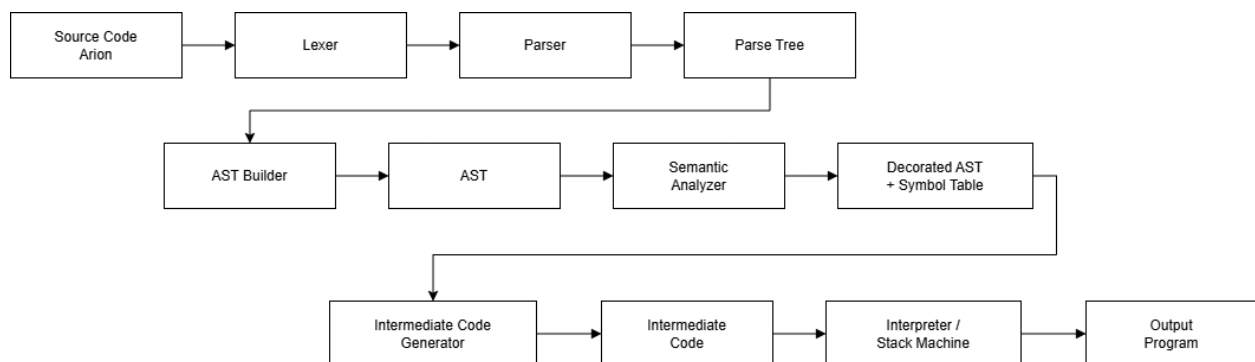
Program diimplementasikan menggunakan bahasa **C++17** dengan bantuan Standard Library seperti `std::vector`, `std::string`, `std::ostringstream`, `std::variant`, `std::unordered_map`, dan `exception handling`. C++17 digunakan karena mendukung pemodelan struktur data compiler secara modular, seperti AST, Symbol Table, Intermediate Code, Runtime Value, dan Stack Machine.

Proses kompilasi program dibantu menggunakan **Makefile** agar seluruh file sumber dapat dibangun secara konsisten. Program dijalankan melalui terminal dengan input berupa source code Arion, parse tree, atau laporan Decorated AST yang sudah memiliki Symbol Table. Output program terdiri dari Decorated AST, Symbol Table, Semantic Diagnostics, Intermediate Code, dan hasil eksekusi program apabila tidak terdapat error.

B. Gambaran Umum Arsitektur

Program pada Milestone 4 dirancang sebagai kelanjutan dari pipeline compiler pada milestone sebelumnya. Pada Milestone 1 sampai Milestone 3, source code Arion diproses melalui lexer, parser, AST builder, dan semantic analyzer hingga menghasilkan **Decorated AST** beserta **Symbol Table**. Pada Milestone 4, kedua hasil tersebut digunakan sebagai input utama untuk menghasilkan Intermediate Code dan menjalankannya menggunakan interpreter.

Secara umum, alur program adalah sebagai berikut.



Arsitektur ini dibuat modular agar setiap tahap memiliki tanggung jawab yang jelas. Lexer dan parser bertanggung jawab terhadap analisis sintaks, semantic analyzer bertanggung jawab terhadap validasi semantik, sedangkan Intermediate Code Generator dan Interpreter bertanggung jawab terhadap proses eksekusi program.

C. Struktur Program

Pada Milestone 4, struktur program diperluas dengan menambahkan folder `src/milestone4/`. Folder ini berisi modul-modul yang berhubungan dengan pembentukan Intermediate Code, pengelolaan konteks code generation, runtime value, stack machine, interpreter, dan runtime error.

Struktur utama program adalah sebagai berikut.

```
ISL-Tubes-IF2224-2026/
├── src/
│   ├── main.cpp
│   ├── token.h
│   ├── lexer.h / lexer.cpp
│   ├── parser.h / parser.cpp
│   ├── parse_tree.h / parse_tree.cpp
│   ├── ast.h / ast.cpp
│   ├── ast_builder.h / ast_builder.cpp
│   ├── symbol_table.h / symbol_table.cpp
│   ├── semantic_analyzer.h / semantic_analyzer.cpp
│   └── milestone4/
│       ├── intermediate_code.h / intermediate_code.cpp
│       ├── codegen_context.h / codegen_context.cpp
│       ├── code_generator.h
│       ├── code_generator_core.cpp
│       ├── code_generator_expr.cpp
│       ├── code_generator_call_io.cpp
│       ├── interpreter.h
│       ├── interpreter_core.cpp
│       ├── interpreter_opr.cpp
│       ├── runtime_stack.h / runtime_stack.cpp
│       ├── runtime_value.h / runtime_value.cpp
│       └── arion_runtime_error.h
└── Makefile
```

Secara umum, modul program dapat dibagi menjadi dua kelompok. Kelompok pertama adalah modul dari milestone sebelumnya yang bertugas menghasilkan Decorated AST dan Symbol Table. Kelompok kedua adalah modul Milestone 4 yang bertugas mengubah Decorated AST menjadi Intermediate Code dan menjalankan instruksi tersebut.

Tabel Modul-Modul Program

Modul	Tanggung Jawab
-------	----------------

<code>main.cpp</code>	Mengatur alur utama program, mulai dari membaca input, menjalankan tahap analisis, membangkitkan Intermediate Code, menjalankan interpreter, sampai menulis output.
<code>token.h</code>	Mendefinisikan jenis token dan struktur token yang digunakan oleh lexer dan parser.
<code>lexer.h / lexer.cpp</code>	Mengubah source code Arion menjadi daftar token.
<code>parser.h / parser.cpp</code>	Membentuk parse tree dari token menggunakan parser.
<code>parse_tree.h / parse_tree.cpp</code>	Menyimpan dan membaca representasi parse tree.
<code>ast.h / ast.cpp</code>	Mendefinisikan struktur AST dan fungsi untuk menampilkan AST atau Decorated AST.
<code>ast_builder.h / ast_builder.cpp</code>	Mengubah parse tree menjadi AST yang lebih ringkas.
<code>symbol_table.h / symbol_table.cpp</code>	Mengelola tabel simbol, scope, identifier, tipe data, dan alamat simbol.
<code>semantic_analyzer.h / semantic_analyzer.cpp</code>	Melakukan semantic analysis dan menghasilkan Decorated AST, Symbol Table, serta Semantic Diagnostics.
<code>intermediate_code.h / intermediate_code.cpp</code>	Mendefinisikan bentuk instruksi Intermediate Code, opcode, operation code, serta fungsi untuk mencetak Intermediate Code.
<code>codegen_context.h / codegen_context.cpp</code>	Menyimpan konteks pembangkitan kode, seperti daftar instruksi, lexical level aktif, entry point subprogram, dan daftar call patch yang perlu diselesaikan setelah target subprogram diketahui.
<code>code_generator.h</code>	Menyediakan antarmuka utama untuk proses Intermediate Code Generation.
<code>code_generator_core.cpp</code>	Menangani implementasi utama code generation, termasuk program, statement, expression, call, I/O, l-value, assignment, control flow, subprogram, dan patching instruksi.
<code>code_generator_expr.cpp</code>	Unit kompilasi pendukung untuk modul code generator. Implementasi utama expression dipusatkan pada <code>code_generator_core.cpp</code> agar mengikuti konvensi l-value

	dan frame yang sama.
<code>code_generator_call_io.cpp</code>	Unit kompilasi pendukung untuk modul code generator. Implementasi utama call dan I/O dipusatkan pada <code>code_generator_core.cpp</code> agar konsisten dengan mekanisme frame, parameter, dan l-value.
<code>runtime_value.h / runtime_value.cpp</code>	Merepresentasikan nilai yang hidup saat runtime, seperti integer, real, boolean, char, string, dan uninitialized value.
<code>runtime_stack.h / runtime_stack.cpp</code>	Mengelola stack runtime, stack frame, slot memori, push/pop value, serta akses address.
<code>interpreter.h</code>	Mendefinisikan class interpreter dan fungsi eksekusi Intermediate Code.
<code>interpreter_core.cpp</code>	Menjalankan instruksi utama seperti INT, LIT, LOD, STO, JMP, JPC, CAL, dan RET.
<code>interpreter_opr.cpp</code>	Menjalankan instruksi OPR, seperti operasi aritmatika, relasional, logika, output, dan input.
<code>arion_runtime_error.h</code>	Mendefinisikan error runtime seperti stack underflow, stack overflow, invalid jump, division by zero, dan runtime type error.
<code>Makefile</code>	Mengatur proses kompilasi seluruh file sumber agar program dapat dibangun secara konsisten.

Dengan pembagian ini, program menjadi lebih mudah dipahami karena setiap modul memiliki tanggung jawab yang jelas. Tahap front-end tetap menangani validasi source code, sedangkan folder `milestone4` berfokus pada tahap back-end, yaitu pembentukan Intermediate Code dan eksekusi program.

D. Fungsi/Kelas Utama

Pada Milestone 4, fungsi dan kelas utama dapat dibagi menjadi tiga bagian besar, yaitu komponen Intermediate Code, komponen Code Generator, dan komponen Interpreter. Komponen Intermediate Code digunakan untuk merepresentasikan instruksi. Komponen Code Generator digunakan untuk mengubah Decorated AST menjadi daftar instruksi. Komponen Interpreter digunakan untuk menjalankan instruksi tersebut menggunakan Stack Machine.

1. Komponen Intermediate Code

Komponen ini berada pada file `intermediate_code.h` dan `intermediate_code.cpp`. Bagian ini mendefinisikan bentuk instruksi yang akan digunakan sebagai hasil translasi dari Decorated AST.

Komponen	Tipe	Peran
OpCode	Enum class	Mendefinisikan instruksi utama seperti <code>LIT</code> , <code>LOD</code> , <code>STO</code> , <code>CAL</code> , <code>INT</code> , <code>JMP</code> , <code>JPC</code> , <code>OPR</code> , dan <code>RET</code> .
OprCode	Enum class	Mendefinisikan operasi khusus pada instruksi <code>OPR</code> , seperti <code>NEG</code> , <code>ADD</code> , <code>SUB</code> , <code>MUL</code> , <code>DIV</code> , <code>MOD</code> , <code>EQL</code> , <code>GTR</code> , <code>WRT</code> , dan <code>WRTLN</code> .
Instruction	Struct	Merepresentasikan satu baris Intermediate Code yang berisi opcode, level, argumen, literal text, dan comment.

Bentuk dasar instruksi dapat digambarkan sebagai berikut.

```
struct Instruction {
    OpCode op;
    int level;
    int arg;
    std::vector<int> extraArgs;
    std::string literalText;
    std::string comment;
};
```

Field `op` menunjukkan jenis instruksi, `level` digunakan untuk akses runtime berdasarkan scope atau frame, sedangkan `arg` digunakan untuk menyimpan nilai operand seperti alamat variabel, target jump, ukuran memori, atau nomor operasi. Field `extraArgs` digunakan untuk menyimpan operand tambahan pada instruksi yang membutuhkan lebih dari satu argumen, seperti `CAL`, `RET`, `IXA`, `WRTLN`, dan `READLN`. Field `literalText` digunakan untuk menyimpan literal tertentu seperti string atau char, sedangkan `comment` digunakan agar output Intermediate Code lebih mudah dibaca.

Selain struktur data, modul ini juga menyediakan fungsi `opcodeToString`, `oprCodeToString`, `renderInstruction`, dan `renderIntermediateCode`. Fungsi-fungsi ini digunakan untuk mengubah daftar instruksi menjadi output teks yang dapat ditampilkan pada laporan program.

2. Komponen Code Generator

Komponen Code Generator digunakan untuk mengubah Decorated AST dan Symbol Table menjadi daftar Intermediate Code. Proses ini dimulai dari fungsi `generateIntermediateCode`.

Fungsi / Kelas	Peran
<code>generateIntermediateCode</code>	Fungsi utama untuk membangkitkan Intermediate Code dari Decorated AST dan Symbol Table.
<code>generateProgram</code>	Memproses root program dan menghasilkan instruksi awal sampai akhir program.
<code>generateStatement</code>	Mengarahkan proses pembangkitan kode berdasarkan jenis statement.
<code>generateExpression</code>	Membentuk instruksi untuk expression, literal, identifier, variable, unary expression, binary expression, dan call.
<code>generateCall</code>	Menangani pemanggilan procedure/function dan operasi I/O.
<code>CodeGenContext</code>	Menyimpan konteks selama proses code generation berjalan.
<code>CodeGenContext::emit</code>	Menambahkan instruksi baru ke daftar Intermediate Code.

<code>CodeGenContext::patch</code>	Mengubah target instruksi jump setelah alamat tujuan diketahui.
<code>CodeGenContext::bindSubprogramEntry</code>	Menyimpan entry point procedure/function.

`CodeGenContext` menjadi komponen penting karena pembangkitan Intermediate Code membutuhkan konteks sementara selama proses translasi AST berjalan. Dalam implementasi aktual, context menyimpan daftar instruksi, lexical level yang sedang aktif, entry point subprogram, serta daftar call patch untuk pemanggilan subprogram yang targetnya belum diketahui saat instruksi pertama kali dibuat.

Alamat runtime dan ukuran frame tidak disimpan oleh `CodeGenContext`, melainkan dihitung dan dinormalisasi terlebih dahulu melalui `normalizeRuntimeLayout`, kemudian disimpan pada Symbol Table. Dengan demikian, code generator cukup membaca informasi layout dari Symbol Table ketika membangkitkan instruksi.

Saat generator menemukan deklarasi variabel atau parameter, generator akan mengalokasikan alamat runtime untuk simbol tersebut. Saat generator menemukan assignment, ekspresi di sisi kanan akan dibangkitkan terlebih dahulu hingga nilainya berada di stack, lalu instruksi `STO` digunakan untuk menyimpan nilai tersebut ke alamat variabel tujuan.

Sebagai contoh:

$$x := 10$$

dapat diterjemahkan menjadi:

```
LIT 0 10
STO 0 alamat_x
```

Untuk ekspresi biner seperti:

$$y := x * 2$$

generator akan menghasilkan instruksi untuk memuat nilai x , memuat literal 2, melakukan operasi perkalian, lalu menyimpan hasilnya ke y .

```
LOD 0 alamat_x
LIT 0 2
OPR 0 4
STO 0 alamat_y
```

Proses ini mengikuti prinsip bahwa Intermediate Code dibentuk sebagai daftar instruksi linear dari Decorated AST. Spesifikasi Milestone 4 juga menekankan bahwa Intermediate Code Generator bertugas mengubah Decorated AST menjadi langkah-langkah instruksi yang akan dijalankan oleh interpreter.

3. Komponen Interpreter dan Runtime

Komponen Interpreter digunakan untuk menjalankan Intermediate Code. Bagian ini berada pada `interpreter.h`, `interpreter_core.cpp`, `interpreter_opr.cpp`, `runtime_stack.h`, `runtime_stack.cpp`, `runtime_value.h`, dan `runtime_value.cpp`.

Komponen	Tipe	Peran
RuntimeValue	Class	Menyimpan nilai runtime dan tipe nilainya.
RuntimeValueKind	Enum class	Membedakan jenis nilai runtime seperti integer, real, boolean, char, string, dan uninitialized.
RuntimeStack	Class	Mengelola stack, slot memori, stack frame, push/pop, dan akses address.
StackFrame	Struct	Menyimpan informasi frame seperti base address, static link, dynamic link, return address, dan slot count.
Interpreter	Class	Menjalankan daftar instruksi Intermediate Code.

<code>InterpreterResult</code>	<code>Struct</code>	Menyimpan output program dan kondisi stack akhir setelah eksekusi.
<code>ArionRuntimeError</code>	<code>Class</code>	Menjadi basis error runtime pada interpreter.

Beberapa method penting pada interpreter dan runtime stack adalah sebagai berikut.

Fungsi / Method	Peran
<code>runIntermediateCode</code>	Fungsi utama untuk menjalankan daftar instruksi.
<code>Interpreter::load</code>	Memuat daftar instruksi ke interpreter.
<code>Interpreter::instructionPointer</code>	Mengambil posisi instruksi yang sedang dijalankan.
<code>Interpreter::setInstructionPointer</code>	Mengubah posisi eksekusi, terutama untuk <code>JMP</code> dan <code>JPC</code> .
<code>Interpreter::advance</code>	Memajukan eksekusi ke instruksi berikutnya.
<code>executeInstruction</code>	Mengeksekusi satu instruksi berdasarkan opcode.
<code>executeOprInstruction</code>	Mengeksekusi operasi khusus pada instruksi <code>OPR</code> .
<code>RuntimeStack::pushValue</code>	Memasukkan nilai ke stack.
<code>RuntimeStack::popValue</code>	Mengambil nilai teratas dari stack.
<code>RuntimeStack::readAt</code>	Membaca nilai dari alamat tertentu.

<code>RuntimeStack::writeAt</code>	Menulis nilai ke alamat tertentu.
<code>RuntimeStack::pushFrame</code>	Membuat stack frame baru saat subprogram dipanggil.
<code>RuntimeStack::popFrame</code>	Menghapus stack frame ketika subprogram selesai.

Interpreter bekerja dengan model Stack Machine. Setiap operasi mengambil operand dari stack, lalu memasukkan hasilnya kembali ke stack. Guidebook Intermediate Code juga menjelaskan bahwa Intermediate Code Arion dirancang berbasis Stack Machine dengan manajemen memori dan eksekusi instruksi.

E. Alur Kerja Program

Program dijalankan melalui terminal dengan satu file input dan satu file output opsional. Input dapat berupa source code Arion, parse tree, atau laporan Decorated AST yang sudah dilengkapi Symbol Table. Dengan rancangan ini, program dapat diuji dari awal pipeline maupun langsung dari hasil milestone sebelumnya.

Alur kerja program adalah sebagai berikut.

1. Validasi argumen

Program membaca argumen CLI. Tanpa flag berarti mode artifact Decorated AST, `--source` berarti source code, dan `--parse-tree` berarti parse tree.

2. Pembacaan file input

Program membaca isi file input sesuai mode yang dipilih.

3. Pemrosesan mode default

Jika tanpa flag, program membaca artifact berisi Decorated AST, Symbol Table, dan Semantic Diagnostics.

4. Pemrosesan source code atau parse tree

Jika menggunakan mode default, program membaca artifact yang berisi Decorated AST, Symbol Table, dan Semantic Diagnostics. Jika menggunakan `--source`,

program menjalankan lexer dan parser untuk menghasilkan parse tree. Jika menggunakan `--parse-tree`, program langsung membaca input sebagai parse tree.

5. **Pembentukan AST**

Parse tree dikonversi menjadi AST menggunakan AST builder agar struktur program menjadi lebih ringkas dan mudah diproses.

6. **Semantic analysis**

AST diproses oleh semantic analyzer untuk menghasilkan Decorated AST, Symbol Table, dan Semantic Diagnostics.

7. **Pemeriksaan semantic diagnostics**

Jika terdapat semantic error, program tetap menampilkan hasil analisis, tetapi tidak melanjutkan ke tahap Intermediate Code Generation.

8. **Intermediate Code Generation**

Jika semantic analysis berhasil, Decorated AST dan Symbol Table digunakan untuk menghasilkan Intermediate Code berupa instruksi seperti `INT`, `LIT`, `LOD`, `STO`, `JMP`, `JPC`, `CAL`, `OPR`, dan `RET`.

9. **Eksekusi Interpreter**

Intermediate Code dijalankan oleh interpreter menggunakan model Stack Machine. Interpreter membaca instruksi satu per satu, mengelola stack, memori, dan instruction pointer.

10. **Pembentukan output akhir**

Program menampilkan Decorated AST, Symbol Table, Semantic Diagnostics, Intermediate Code, dan Program Output. Jika terjadi error pada tahap Milestone 4, program menampilkan pesan error yang sesuai.

F. Implementasi Pembacaan dan Validasi Decorated AST

Masukan resmi tahap ini adalah Decorated AST, bukan source code Arion. Program menggunakan struktur FrontendArtifact yang menggabungkan tiga komponen, yaitu Decorated AST, Symbol Table, dan Semantic Diagnostics. Ketiga komponen tersebut disimpan dalam format teks dengan penanda bagian sebagai berikut:

```
----- Decorated AST -----
...
----- Symbol Table -----
...
----- Semantic Diagnostics -----
...
```

Pemrosesan artifact diimplementasikan pada `frontend_artifact.h` dan `frontend_artifact.cpp`. Fungsi `parseFrontendArtifact` memisahkan isi berkas berdasarkan penanda bagian, membangun kembali pohon AST melalui `parseRenderedAst`, memuat tabel `tab`, `btab`, dan `atab` melalui `SymbolTable::loadRenderedAll`, lalu membaca setiap baris diagnostic sebagai error atau warning. Dengan pendekatan ini, hasil front-end tidak hanya menjadi tampilan laporan, tetapi menjadi representasi serial yang dapat dibaca kembali oleh program.

Setelah artifact dibaca, fungsi `validateFrontendArtifact` memeriksa konsistensi data sebelum code generation dilakukan. Validasi mencakup:

1. root Decorated AST harus berupa node Program;
2. `tab` dan `btab` tidak boleh kosong;
3. setiap `symbolIndex` pada node AST harus berada dalam rentang `tab`;
4. nama identifier pada AST harus konsisten dengan identifier yang dirujuk pada Symbol Table;
5. lexical level tidak boleh bernilai negatif;
6. reference tipe array harus menunjuk entri `atab` yang valid;
7. reference tipe record dan block procedure/function harus menunjuk entri `btab` yang valid, dan jumlah child untuk node penting seperti assignment, binary expression, if, while, repeat, dan for harus sesuai dengan bentuk AST yang diharapkan.

Program menggunakan mode artifact sebagai mode masukan utama `./arion input-artifact.txt [output.txt]`

Untuk kebutuhan integrasi dan pengujian regresi, program juga menyediakan dua mode tambahan, yaitu `./arion -source input-arion.txt [output.txt]` dan `./arion -parse-tree input-parse-tree.txt [output.txt]`

Mode `-source` menjalankan seluruh pipeline mulai dari lexer sampai interpreter, sedangkan mode `-parse-tree` memulai pipeline dari parse tree. Meskipun demikian, kedua mode tersebut hanya merupakan fasilitas integrasi. Alur resmi back-end tetap menggunakan Decorated AST, Symbol Table, dan Semantic Diagnostics sebagai artifact masukan.

Program juga menerapkan semantic gating. Apabila artifact memuat semantic error, Decorated AST, Symbol Table, dan Semantic Diagnostics tetap dituliskan ke output, tetapi Intermediate Code tidak dibangun dan interpreter tidak dijalankan. Keputusan ini mencegah back-end menjalankan program yang maknanya belum valid. Bagian Intermediate Code dan Program Output yang mungkin terdapat pada artifact lama juga tidak dibaca

kembali, sehingga Intermediate Code selalu dibangkitkan ulang dari Decorated AST yang sedang diproses.

G. Implementasi Normalisasi Layout Runtime

Informasi pada Symbol Table hasil semantic analysis belum selalu memiliki layout memori final yang sesuai untuk kebutuhan interpreter. Oleh karena itu, sebelum code generation, program memanggil `normalizeRuntimeLayout` untuk menghitung ulang alamat dan ukuran seluruh data runtime.

Setiap activation frame menggunakan tiga slot logis awal:

```
offset 0 : static link  
offset 1 : dynamic link  
offset 2 : return address  
offset 3 dan seterusnya : parameter, hasil function, dan variabel  
lokal
```

Pada implementasi aktual, static link, dynamic link, dan return address disimpan sebagai metadata pada `StackFrame`, sedangkan vector slots tetap menyediakan layout logis yang dimulai dari ukuran frame tersebut. Akses program ke offset 0 sampai 2 dilarang oleh `validateAddress`. Dengan demikian, instruksi program tidak dapat membaca atau menimpa header frame.

Fungsi `runtimeTypeSize` menghitung jumlah slot yang dibutuhkan suatu tipe. Tipe scalar integer, real, boolean, char, string, subrange, dan enum menggunakan satu slot. Ukuran array diperoleh dari entri `atab`, sedangkan ukuran record dihitung dari penjumlahan ukuran seluruh field secara rekursif. Karena elemen array atau field record dapat kembali berupa array atau record, perhitungan ini mendukung composite value bersarang.

Normalisasi layout dilakukan dengan aturan berikut:

1. Variabel global dialokasikan mulai offset 3 pada global frame.
2. Parameter subprogram dialokasikan mulai offset 3 sesuai urutan deklarasinya.
3. Untuk function, slot hasil function ditempatkan setelah seluruh parameter.
4. Variabel lokal ditempatkan setelah parameter dan slot hasil function.
5. Field record diberi offset relatif mulai dari 0 di dalam record.
6. Ukuran elemen dan ukuran total array dihitung ulang berdasarkan tipe elemen aktual.

7. psze pada btab menyimpan jumlah slot parameter, sedangkan vsze menyimpan slot hasil function dan variabel lokal.

Kolom ref dan typeRef pada tab dibedakan untuk menghindari ambiguitas. ref digunakan sebagai reference block milik procedure atau function, sedangkan typeRef menyimpan reference tipe array atau record. Pemisahan ini penting untuk function yang mengembalikan composite value karena function membutuhkan reference block sekaligus reference tipe hasil.

H. Implementasi Representasi Intermediate Code

Intermediate Code direpresentasikan sebagai vector berisi objek Instruction. Setiap instruksi memiliki opcode, lexical-level difference, operand utama, operand tambahan, literal teks, dan komentar.

```
struct Instruction {
    OpCode op;
    int level;
    int arg;
    std::vector<int> extraArgs;
    std::string literalText;
    std::string comment;
};
```

Field level digunakan oleh instruksi yang mengakses activation frame berdasarkan perbedaan lexical level. Field arg digunakan untuk alamat, target jump, ukuran frame, kode operasi, atau operand utama lain. extraArgs dipakai ketika sebuah instruksi membutuhkan informasi tambahan, misalnya jumlah slot argumen pada CAL, ukuran hasil pada RET, dan batas serta ukuran elemen pada IXA. literalText mempertahankan bentuk literal real, boolean, char, dan string agar tidak kehilangan informasi ketika Intermediate Code dirender ke teks.

Instruksi dasar yang diwajibkan spesifikasi tetap digunakan, seperti:

1. LIT memasukkan literal ke operand stack.
2. LOD memuat nilai scalar dari frame.
3. STO menyimpan nilai scalar ke frame.
4. CAL memanggil procedure atau function.
5. INT menginisialisasi ukuran activation frame.
6. JMP melakukan unconditional jump.
7. JPC melakukan jump apabila kondisi bernilai false.

8. OPR menjalankan operasi aritmatika, relasional, boolean, atau I/O.
9. RET mengakhiri program atau mengembalikan eksekusi ke caller.

Implementasi menambahkan tujuh instruksi untuk mendukung l-value dan composite value:

1. LDA l a memuat alamat pada lexical-level difference l dan offset a.
2. LDI membaca nilai scalar secara tidak langsung dari alamat teratas.
3. STI menyimpan nilai scalar secara tidak langsung ke suatu alamat.
4. BLD n memuat block data sebanyak n slot.
5. BST n menyimpan block data sebanyak n slot.
6. ADI o menambahkan offset field record pada suatu alamat.
7. IXA low high size menghitung alamat elemen array sekaligus memeriksa batas indeks.

Penambahan instruksi tersebut diperlukan karena LOD dan STO saja hanya cukup untuk variabel scalar dengan alamat statis. Akses seperti `values[i].left`, assignment antarecord, parameter record, dan function yang mengembalikan array atau record membutuhkan alamat yang dapat dihitung saat runtime serta operasi block.

Instruksi OPR menggunakan kode operasi NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ, WRT, WRTLN, IDIV, AND, OR, NOT, dan READLN. Kode 1 sampai 14 mempertahankan konvensi dasar spesifikasi, sedangkan IDIV, operasi boolean, dan READLN menjadi extension yang diperlukan untuk fitur bahasa yang diimplementasikan.

Intermediate Code dirender dalam format index OP CODE level operand [operand-tambahan] ; komentar.

Modul `intermediate_code.cpp` juga menyediakan `parseIntermediateCode`. Pada alur utama, hasil generator dirender menjadi teks lalu dibaca kembali sebelum dieksekusi. Proses round-trip ini memastikan bahwa format Intermediate Code yang ditampilkan pada output benar-benar merupakan format yang dapat dibaca oleh interpreter, bukan hanya representasi visual.

I. Implementasi Intermediate Code Generator

Intermediate Code Generator menerima Decorated AST dan Symbol Table melalui fungsi `generateIntermediateCode`. Penelusuran dilakukan secara rekursif mengikuti struktur AST. Pendekatan ini sesuai dengan prinsip DFS-based code generation, tetapi urutan emisi instruksi disesuaikan dengan kebutuhan stack machine. Sebagai contoh, pada binary

expression, operand kiri dibangkitkan terlebih dahulu, dilanjutkan operand kanan, kemudian instruksi OPR.

CodeGenContext menyimpan daftar instruksi, lexical level yang sedang aktif, entry point subprogram, dan daftar call yang targetnya belum diketahui. Method emit menambahkan instruksi, sedangkan patch mengisi target jump setelah alamat tujuan diketahui. Mekanisme patching digunakan pada control flow dan forward call.

1. Pembangkitan Program dan Subprogram

Procedure dan function ditempatkan sebelum body program utama. Apabila program memiliki subprogram, generator terlebih dahulu menghasilkan JMP menuju main program agar subprogram tidak dieksekusi secara langsung. Entry point setiap subprogram kemudian diikat ke symbol index-nya.

Untuk subprogram bersarang, generator kembali menghasilkan jump yang melewati deklarasi subprogram internal menuju body subprogram induk. Rancangan ini meratakan struktur AST menjadi daftar instruksi linear tanpa menghilangkan hubungan lexical scope.

Setiap program atau subprogram diawali dengan INT untuk menginisialisasi frame. Besarnya frame diperoleh dari hasil normalisasi layout ukuran frame = 3 + psze + vsze.

Setelah INT, argumen yang sebelumnya berada pada operand stack dipindahkan ke slot parameter. Parameter composite dipindahkan menggunakan BST, sedangkan parameter scalar menggunakan STI.

2. Pembangkitan Literal, Identifier, dan Ekspresi

Literal diterjemahkan menjadi LIT. Variabel scalar sederhana diterjemahkan menjadi LOD dengan lexical-level difference dan offset dari Symbol Table. Konstanta menggunakan nilai literal yang disimpan pada tab.

Unary expression diterjemahkan sebagai berikut:

-x	-> kode x, OPR NEG
not x	-> kode x, OPR NOT
+x	-> kode x

Binary expression membangkitkan operand kiri, operand kanan, lalu operasi yang sesuai. Operator +, -, *, /, div, mod, and, or, ==, !=, <, >=, >, dan <= dipetakan ke kode OPR masing-masing. Interpreter akan menghasilkan integer apabila kedua operand operasi aritmatika adalah integer, dan real apabila salah satu operand berupa real.

3. Pembangkitan Assignment dan L-Value

Untuk assignment scalar sederhana, expression di sisi kanan dibangkitkan terlebih dahulu lalu disimpan menggunakan STO.

```
x := 10  
  
LIT 0 10  
STO 0 alamat_x
```

Untuk target yang mengandung akses indeks, akses field, atau merupakan hasil function, generator membangun alamat target dengan LDA, IXA, dan ADI. Nilai scalar disimpan dengan STI, sedangkan composite value disimpan dengan BST.

Urutan operand pada assignment tidak langsung adalah nilai terlebih dahulu, kemudian alamat. Instruksi STI mengambil alamat dari puncak stack, mengambil nilai di bawahnya, lalu menuliskan nilai tersebut. Konvensi yang sama digunakan oleh BST untuk block data.

4. Pembangkitan Control Flow

Statement if membangkitkan expression kondisi dan JPC dengan target sementara. Apabila terdapat else, generator menambahkan JMP untuk melewati blok else. Kedua target diisi menggunakan patch setelah posisi instruksi diketahui.

Statement while menyimpan index awal kondisi. Setelah body selesai, JMP kembali ke kondisi, sedangkan JPC diarahkan ke instruksi setelah loop.

Statement repeat menempatkan label awal sebelum body. Kondisi dievaluasi setelah body dan JPC kembali ke awal selama kondisi masih false.

Statement for melakukan empat tahap: menginisialisasi counter, membandingkan counter dengan batas akhir menggunakan LEQ atau GEQ, menjalankan body, lalu menambah atau mengurangi counter sebelum kembali ke kondisi.

Statement case membandingkan expression selector dengan setiap label menggunakan EQL. Setiap cabang yang berhasil dijalankan diakhiri dengan JMP menuju akhir statement case.

Pemakaian JMP dan JPC dipilih karena kedua instruksi tersebut cukup untuk merepresentasikan seluruh control flow terstruktur dalam bentuk linear.

J. Implementasi Runtime Value dan Memori Eksekusi

Nilai runtime dimodelkan oleh `RuntimeValue` menggunakan `std::variant`. Jenis nilai yang didukung adalah `Uninitialized`, `Integer`, `Real`, `Boolean`, `Char`, `String`, dan `Address`. Penggunaan `variant` membuat setiap nilai membawa jenis aktualnya sehingga interpreter dapat melakukan validasi tipe saat eksekusi.

`Address` tidak disimpan sebagai integer tunggal, tetapi sebagai pasangan (`frameIndex`, `offset`). Representasi ini mencegah alamat dari suatu frame disalahartikan sebagai alamat global dan memudahkan validasi apakah frame yang dirujuk masih aktif.

Memori eksekusi dikelola oleh `RuntimeStack`. Implementasi memisahkan dua struktur, yaitu `frames_` (kumpulan activation frame untuk variabel, parameter, hasil function, static link, dynamic link, dan return address) dan `operands_` (operand stack untuk literal, hasil sementara, alamat, argumen, dan hasil function). Pemisahan ini dilakukan karena Temporary value milik caller tidak bercampur dengan slot lokal callee. Setiap CAL mencatat `operandBase`, yaitu kedalaman operand stack sebelum argumen call. Ketika RET dijalankan, interpreter memastikan kedalaman operand kembali sama dengan kontrak call tersebut. Pemeriksaan ini dapat mendeteksi operand yang tertinggal atau terambil secara tidak semestinya.

Static link digunakan untuk akses nonlocal berdasarkan lexical scope. Instruksi LOD, STO, LDA, dan CAL membawa lexical-level difference. `RuntimeStack::frameIndexAtDifference` mengikuti static link sebanyak nilai difference tersebut. Dynamic link menunjuk caller aktual dan diverifikasi ketika frame dikeluarkan. Dengan demikian, lexical nesting dan urutan pemanggilan runtime ditangani oleh mekanisme yang berbeda.

Global frame dibuat ketika INT pertama dijalankan. Pemanggilan subprogram membuat frame kosong yang menyimpan static link, dynamic link, return address, dan operand base. Instruksi INT pada entry point subprogram kemudian mengalokasikan slots sesuai ukuran frame yang telah dihitung.

K. Implementasi Interpreter

Interpreter menjalankan Intermediate Code sebagai virtual machine berbasis stack. Proses utama terdapat pada `runIntermediateCode`, yang memuat daftar instruksi, memvalidasi seluruh target JMP, JPC, dan CAL, lalu menjalankan siklus fetch-decode-execute sampai program berhenti.

Pada setiap iterasi, interpreter melakukan hal sebagai berikut:

1. Mengambil instruksi pada instruction pointer.
2. Menentukan opcode instruksi.
3. Menjalankan perubahan pada operand stack, frame, output, atau instruction pointer.
4. Memajukan instruction pointer untuk instruksi biasa atau menggantinya untuk jump dan call.
5. Menangkap runtime error serta menambahkan posisi instruction pointer pada pesan error.

Perilaku instruksi inti adalah sebagai berikut:

1. INT mengalokasikan slots pada frame aktif.
2. LIT memasukkan literal ke operand stack.
3. LOD membaca nilai dari frame dan memasukkannya ke operand stack.
4. STO mengambil nilai dari operand stack dan menyimpannya ke frame.
5. LDA memasukkan alamat tervalidasi ke operand stack.
6. LDI membaca nilai dari alamat pada operand stack.
7. STI menyimpan nilai secara tidak langsung.
8. BLD membaca beberapa slot berurutan dan memasukkannya ke operand stack.
9. BST menyimpan beberapa nilai berurutan ke suatu alamat.
10. ADI menggeser alamat berdasarkan offset field.
11. IXA memeriksa batas array dan menghitung alamat elemen.
12. CAL membuat activation frame dan berpindah ke entry point subprogram.
13. RET mengembalikan hasil function, menghapus frame, dan melanjutkan caller.
14. JMP berpindah tanpa syarat.
15. JPC mengambil kondisi dan berpindah apabila kondisi false.
16. OPR menjalankan operasi aritmatika, relasional, boolean, output, atau input.

Program utama diakhiri oleh RET tanpa caller. Pada kondisi tersebut, interpreter memasuki status halted. Setelah halt, interpreter memeriksa bahwa hanya global frame yang tersisa dan operand stack kosong. Apabila kondisi ini tidak terpenuhi, program dianggap mengalami stack corruption.

L. Implementasi Procedure, Function, Nested Scope, dan Rekursi

Konvensi call yang digunakan adalah:

```
CAL level target argumentSlotCount  
RET 0 resultOffset resultSlotCount
```

Sebelum CAL, caller membangkitkan seluruh argumen ke operand stack sesuai urutan parameter. argumentSlotCount menyatakan jumlah slot, bukan jumlah expression. Perbedaan ini diperlukan karena satu argumen record atau array dapat menggunakan lebih dari satu slot.

CAL menentukan static parent dengan mengikuti static link sebanyak operand level. Dynamic parent selalu merupakan frame caller yang sedang aktif. Return address disimpan sebagai instruction pointer setelah CAL.

Pada awal callee, INT mengalokasikan frame. Argumen kemudian dipindahkan dari operand stack ke slot parameter dalam urutan terbalik agar parameter pertama tetap menerima argumen pertama. Procedure mengakhiri eksekusi dengan RET berukuran hasil nol.

Bahasa Arion tidak menggunakan keyword return. Nilai function diberikan melalui assignment ke nama function. Nama function diperlakukan sebagai l-value khusus yang mengarah ke slot hasil pada frame aktif. Saat RET, interpreter membaca resultSlotCount slot mulai dari resultOffset, menghapus frame callee, lalu memasukkan hasil tersebut kembali ke operand stack caller. Mekanisme ini mendukung function scalar maupun function yang mengembalikan array atau record.

Rekursi didukung karena setiap pemanggilan menghasilkan activation frame baru dengan parameter, variabel lokal, dan hasil function yang terpisah. Nested procedure juga dapat membaca atau menulis variabel pada lexical scope luar menggunakan static link. Contohnya, procedure bersarang dapat mengakses variabel lokal procedure induk dan variabel global pada saat yang sama.

Target call yang belum diketahui ketika instruksi dibangkitkan dicatat sebagai call patch berdasarkan symbol index. Setelah deklarasi subprogram terkait memperoleh entry point, seluruh call tersebut diperbarui. Setelah code generation selesai, validateCallPatches memastikan tidak ada target call yang belum terselesaikan.

M. Implementasi Array, Record, dan Composite Value

Array dan record disimpan secara flat sebagai beberapa slot berurutan pada activation frame. Pendekatan ini membuat assignment, parameter, dan return value composite dapat menggunakan operasi block tanpa membutuhkan object heap terpisah.

Untuk array, atab menyimpan batas bawah, batas atas, tipe elemen, reference elemen, ukuran elemen, dan ukuran total. Akses array dibangkitkan dengan urutan:

1. LDA ... (alamat awal array)
2. kode expression i (nilai indeks)
3. IXA low high elsz (validasi dan hitung alamat elemen)

Interpreter memeriksa $\text{low} \leq i \leq \text{high}$. Jika valid, offset ditambah dengan $(i - \text{low}) * \text{elementSize}$

Karena elementSize dapat lebih dari satu, instruksi yang sama dapat digunakan untuk array of record dan array bersarang.

Untuk record, setiap field memiliki offset relatif pada tab. Akses field dilakukan menggunakan ADI. Akses `values[i].left` berarti generator terlebih dahulu menghitung alamat `values[i]` dengan IXA, lalu menambahkan offset field left dengan ADI.

Pembacaan composite value menggunakan BLD size, sedangkan penulisannya menggunakan BST size. Kedua instruksi tersebut digunakan untuk:

1. Assignment array atau record.
2. Parameter composite.
3. Pemindahan nilai composite ke slot parameter callee.
4. Return value composite dari function.

Pemilihan penyimpanan flat mempermudah validasi batas dan penyalinan nilai. Konsekuensinya, assignment composite menggunakan copy-by-value. Perilaku ini konsisten dengan parameter normal pada implementasi saat ini.

N. Implementasi Input dan Output

Built-in `write`, `writeln`, dan `readln` ditangani pada code generator sebagai pemanggilan khusus, bukan CAL menuju subprogram biasa.

Untuk `write`, setiap expression argumen dibangkitkan lalu dieksekusi dengan OPR WRT. Untuk `writeln`, argumen terakhir menggunakan OPR WRTLN agar newline ditambahkan setelah nilai terakhir. `writeln` tanpa argumen menghasilkan newline tanpa mengambil operand.

Program tidak menambahkan separator otomatis di antara argumen. Oleh sebab itu, `writeln(10, 20)` akan menghasilkan 1020.

Jika pemisah diperlukan, pemisah harus dituliskan sebagai argumen string, misalnya `writeln(10, ' ', 20)`.

`readln` menerima satu atau lebih variabel scalar. Generator memuat alamat setiap variabel dengan LDA, lalu menghasilkan OPR READLN dengan kode tipe:

- 1 = integer, subrange, atau enum
- 2 = real
- 3 = boolean
- 4 = char
- 5 = string

Interpreter membaca input berbasis token yang dipisahkan whitespace. Nilai kemudian dikonversi sesuai kode tipe dan ditulis ke alamat tujuan. Apabila token tidak sesuai tipe, interpreter menghasilkan `RuntimeTypeError`.

O. Implementasi Error Handling dan Runtime Safety

Penanganan error dilakukan pada setiap batas tahap pipeline.

1. Kesalahan argumen command line dan pembacaan file menghentikan program dengan exit code nonzero.
2. Lexical error membatalkan parsing.
3. Syntax error, kegagalan AST builder, dan artifact yang tidak valid ditangkap sebagai exception.
4. Semantic error membatalkan code generation dan interpreter.
5. Code generator menolak node, operator, reference, atau call yang tidak konsisten.
6. Interpreter memvalidasi target jump dan call sebelum eksekusi.
7. Runtime error menghentikan program secara terkendali dan menyertakan instruction pointer.

Hierarchy runtime error terdiri dari `ArionRuntimeError` sebagai base class dan beberapa turunan:

1. `StackOverflowError`
2. `StackUnderflowError`
3. `InvalidJumpTargetError`
4. `DivisionByZeroError`

5. `RuntimeTypeError`
6. `BoundsError`
7. `NumericOverflowError`
8. `NumericUnderflowError`
9. `InvalidMemoryError`
10. `StackCorruptionError`.

Interpreter melakukan pemeriksaan keamanan sebagai berikut:

1. Stack Overflow dan Underflow

Jumlah activation frame dibatasi maksimal 1000 secara default. Rekursi tanpa akhir akan dihentikan ketika batas ini tercapai. Operasi pop dan peek pada operand stack selalu memeriksa apakah jumlah operand mencukupi. Global frame tidak dapat dikeluarkan.

2. Perlindungan Frame dan Memori

Setiap alamat memuat frame index dan offset. Interpreter menolak reference ke frame yang sudah tidak aktif, offset negatif, offset melewati ukuran frame, dan akses ke tiga slot header. Pembacaan slot yang belum diinisialisasi juga menghasilkan error.

3. Validasi Call dan Return

CAL memeriksa bahwa jumlah slot argumen tidak melebihi operand stack. Target call harus menunjuk entry point yang diawali INT atau jump menuju body subprogram. RET memeriksa operand depth, dynamic link, return address, dan ukuran hasil. Setelah program berakhir, jumlah frame dan operand harus seimbang.

4. Bounds Checking

IXA memeriksa indeks dinamis terhadap batas array. Pemeriksaan ini tetap dilakukan saat runtime meskipun semantic analyzer telah memeriksa indeks literal, karena nilai indeks yang berasal dari variabel baru diketahui ketika program dijalankan.

5. Validasi Operasi Numerik

Operasi integer dilakukan dengan perantara `int64_t`, kemudian diperiksa agar tetap berada dalam rentang signed 32-bit. Real operation diperiksa menggunakan `std::isfinite` dan klasifikasi floating point untuk mendeteksi overflow dan underflow. Pembagian dan modulo dengan nol ditolak sebelum operasi dilakukan.

6. Validasi Tipe Runtime

Setiap operasi memeriksa jenis operand aktual. Operasi aritmatika hanya menerima nilai numeric, div dan mod hanya menerima integer, operasi boolean hanya menerima boolean,

serta JPC menerima boolean atau representasi integer nol. Perbandingan numeric mendukung kombinasi integer dan real, sedangkan char, string, dan boolean harus dibandingkan dengan jenis yang sama.

Pesan error dibungkus dengan instruction pointer. Contohnya adalah “[ERROR] ip 6: array index 3 outside 1..2”

Informasi tersebut mempermudah pencocokan error dengan baris Intermediate Code yang sedang dieksekusi.

P. Implementasi Output Akhir dan Justifikasi Rancangan

Jika seluruh tahap berhasil, program menghasilkan lima bagian:

1. Decorated AST.
2. Symbol Table.
3. Semantic Diagnostics.
4. Intermediate Code.
5. Program Output.

Penyusunan output dalam satu laporan teks memudahkan penelusuran hubungan antara node AST, symbol reference, alamat runtime, instruksi yang dihasilkan, dan hasil program. Intermediate Code selalu dirender sebelum interpreter dijalankan. Interpreter kemudian menjalankan hasil parsing ulang dari teks Intermediate Code tersebut.

Secara keseluruhan, beberapa keputusan implementasi utama adalah:

1. Stack-machine intermediate code. TAC digunakan sebagai konsep perataan operasi, tetapi representasi final yang dihasilkan dan dieksekusi adalah instruksi stack machine.
2. Artifact front-end yang dapat dibaca kembali.
3. Normalisasi layout sebelum code generation. Alamat dan ukuran data tidak bergantung pada keadaan sementara semantic analyzer.
4. Pemisahan activation frame dan operand stack. Temporary caller tetap utuh ketika callee dijalankan, serta kontrak call-return dapat diverifikasi.
5. Static link dan dynamic link terpisah. Static link menyelesaikan lexical scope, sedangkan dynamic link merepresentasikan hubungan caller-callee.
6. Alamat terstruktur. Pasangan frame index dan offset lebih aman dibandingkan alamat integer mentah.

7. Composite value menggunakan block operation. Array dan record dapat ditangani secara umum pada assignment, parameter, dan return value.
8. Validasi berlapis. Error semantik dihentikan sebelum code generation, sedangkan kondisi yang hanya diketahui saat eksekusi diperiksa oleh interpreter.

Arsitektur tersebut memungkinkan program menjalankan fitur scalar, arithmetic, boolean, control flow, procedure/function, recursion, nested scope, I/O, array, record, composite assignment, parameter composite, dan return value composite dalam satu konvensi runtime yang konsisten.

Pengujian

Untuk memastikan bahwa tahap Intermediate Code Generation dan Interpreter pada Milestone 4 berjalan dengan benar, dilakukan serangkaian pengujian menggunakan berbagai skenario input bahasa Arion. Pengujian ini mencakup Intermediate Code *generation*, eksekusi program dengan Stack Machine Interpreter, pemanggilan procedure/function, control flow, array dan record, I/O, penanganan semantic error, runtime error, serta pembacaan artifact Decorated AST.

Pengujian 1

Diberikan input sebagai berikut:

```
program MemoryAssignment;

var
  a, b: integer;

begin
  a := 5;
  b := a
end.
```

Dihasilkan output Intermediate Code sebagai berikut:

```
0 INT 0 5
1 LIT 0 5
2 STO 0 3
3 LOD 0 3
4 STO 0 4
5 RET 0 0
```

Pengujian ini memastikan bahwa program dapat men-*generate* Intermediate Code dasar dari assignment variabel. Karena program tidak memiliki write atau writeln, bagian yang diperiksa adalah instruksi Intermediate Code, terutama alokasi memori dengan INT, pemuatan literal dengan LIT, penyimpanan nilai dengan STO, dan pembacaan nilai dengan LOD.

Pengujian 2

Diberikan input sebagai berikut:

```
program CallsRecursion;

var
  base: integer;
```

```

function fact(n: integer): integer;
begin
    if n <= 1 then
        fact := 1
    else
        fact := n * fact(n - 1)
    end;

procedure outer(x: integer);
var
    local: integer;

    procedure addbase(y: integer);
    begin
        local := y + base
    end;

begin
    local := x;
    addbase(local);
    writeln(local)
end;

procedure ping;
begin
    writeln(1)
end;

begin
    base := 2;
    outer(fact(4));
    ping
end.

```

Dihasilkan output sebagai berikut:

```

26
1

```

Pengujian ini memastikan bahwa interpreter dapat menjalankan pemanggilan function dan procedure menggunakan mekanisme stack frame. Kasus ini juga menguji recursion melalui function `fact`, akses variabel dari lexical scope luar, nested procedure, serta procedure tanpa parameter yang dipanggil tanpa tanda kurung.

Pengujian 3

Diberikan input sebagai berikut:

```

program Composites;

```

```

type
  Pair == record
    left: integer;
    right: integer
  end;
  PairArray == array[1..2] of Pair;
  Inner == record
    first: integer;
    second: integer
  end;
  Outer == record
    inner: Inner;
    suffix: integer
  end;
  OuterArray == array[1..1] of Outer;
  Box == record
    items: PairArray;
    tag: integer
  end;

var
  values: PairArray;
  copied: Pair;
  boxed: Box;
  nested: OuterArray;
  returned: PairArray;
  i: integer;

procedure printPair(p: Pair);
begin
  writeln(p.left, p.right)
end;

procedure printSecond(items: PairArray);
begin
  writeln(items[2].left, items[2].right)
end;

function makePair(a, b: integer): Pair;
begin
  makePair.left := a;
  makePair.right := b
end;

begin
  i := 2;
  values[1].left := 10;
  values[1].right := 20;

```



```

values[i] := values[1];
copied := values[i];
printPair(copied);

values[2].left := 3;
values[2].right := 4;
writeln(values[2].left, values[2].right);

boxed.items[2].left := 5;
boxed.items[2].right := 6;
boxed.tag := 7;
writeln(boxed.items[2].left, boxed.items[2].right, boxed.tag);

nested[1].inner.first := 1;
nested[1].inner.second := 2;
nested[1].suffix := 34;
writeln(nested[1].inner.first, nested[1].inner.second,
nested[1].suffix);

values[1].left := 0;
values[1].right := 0;
values[2].left := 5;
values[2].right := 6;
printSecond(values);

copied := makePair(3, 4);
writeln(copied.left, copied.right);

returned[1] := makePair(5, 6);
writeln(returned[1].left, returned[1].right)
end.

```

Dihasilkan output sebagai berikut:

```

1020
34
567
1234
56
34
56

```

Pengujian ini memastikan bahwa program dapat menangani tipe composite seperti array dan record. Kasus ini mencakup akses field record, array of record, record yang memiliki field berupa array, nested record, penyalinan nilai composite, pengiriman composite sebagai parameter, serta function yang mengembalikan record.

Pengujian 4

Diberikan input sebagai berikut:

```
program ControlScalars;  
  
var  
  i, total: integer;  
  ok: boolean;  
  r: real;  
  
begin  
  total := 0;  
  for i := 1 to 3 do  
    begin  
      total := total + i  
    end;  
  while total < 8 do  
    begin  
      total := total + 1  
    end;  
  repeat  
    total := total - 1  
  until total == 7;  
  case total of  
    6: total := 0;  
    7: total := total + 3  
  end;  
  ok := (total == 10) and not false;  
  r := total / 4;  
  r := r + 0.5;  
  if ok then  
    writeln(total, r)  
  end.  
end.
```

Dihasilkan output sebagai berikut:

```
103
```

Pengujian ini memastikan bahwa struktur control flow dapat diterjemahkan menjadi instruksi jump pada Intermediate Code dan dieksekusi dengan benar oleh interpreter. Kasus ini mencakup `for`, `while`, `repeat-until`, `case`, operasi boolean, serta operasi scalar pada integer dan real.

Pengujian 5

Diberikan input sebagai berikut:

```
program InputOutput;
```

```

var
  number: integer;
  ratio: real;
  letter: char;
  word: string;
  flag: boolean;

begin
  readln(number, ratio, letter, word, flag);
  write(number, ratio, letter, word);
  writeln(flag)
end.

```

Dihasilkan output sebagai berikut:

```
423.5Ahellotrue
```

Pengujian ini memastikan bahwa built-in procedure `readln`, `write`, dan `writeln` berjalan dengan benar. Nilai dari input dibaca sesuai tipe variabel tujuan, kemudian hasilnya dicetak kembali melalui output buffer interpreter.

Pengujian 6

Diberikan input sebagai berikut:

```

program DefiniteAssignment;

var
  x, y: integer;

begin
  if true then
    x := 1;
    y := x
  end.

```

Dihasilkan output (build/semantic-error.txt) sebagai berikut:

```

----- Decorated AST -----

Program(DefiniteAssignment) [type=none, sym=38, level=0]
|-- Declarations [type=none, level=0]
|   |-- VarDecl(x) [type=integer, sym=39, level=0]
|   |   \-- NamedType(integer) [type=integer, sym=22, level=0]
|   \-- VarDecl(y) [type=integer, sym=40, level=0]
|       \-- NamedType(integer) [type=integer, sym=22, level=0]
\-- CompoundStmt [type=none, level=1]
    |-- IfStmt [type=none, level=1]
    |   |-- Literal(true) [type=none, level=1]

```

```

|    |-- AssignStmt [type=none, level=1]
|    |    |-- Variable(x) [type=integer, sym=39, level=1]
|    |    |-- Literal(1) [type=integer, level=1]
|-- AssignStmt [type=none, level=1]
|    |-- Variable(y) [type=integer, sym=40, level=1]
|    |-- Identifier(x) [type=integer, sym=39, level=1]

```

----- Symbol Table -----

tab

idx	identifier	obj	type	ref	tref	nrm	lev
adr	link	init	value				
0	<dummy>	none	none	0	0	1	0
0	0	1					
1	and	reserved	none	0	0	1	0
0	0	1					
2	array	reserved	none	0	0	1	0
0	1	1					
3	begin	reserved	none	0	0	1	0
0	2	1					
4	case	reserved	none	0	0	1	0
0	3	1					
5	const	reserved	none	0	0	1	0
0	4	1					
6	div	reserved	none	0	0	1	0
0	5	1					
7	downto	reserved	none	0	0	1	0
0	6	1					
8	do	reserved	none	0	0	1	0
0	7	1					
9	else	reserved	none	0	0	1	0
0	8	1					
10	end	reserved	none	0	0	1	0
0	9	1					
11	for	reserved	none	0	0	1	0
0	10	1					
12	function	reserved	none	0	0	1	0
0	11	1					
13	if	reserved	none	0	0	1	0
0	12	1					
14	mod	reserved	none	0	0	1	0
0	13	1					
15	not	reserved	none	0	0	1	0
0	14	1					
16	of	reserved	none	0	0	1	0

0	15	1						
17	or		reserved	none	0	0	1	0
0	16	1						
18	procedure		reserved	none	0	0	1	0
0	17	1						
19	program		reserved	none	0	0	1	0
0	18	1						
20	record		reserved	none	0	0	1	0
0	19	1						
21	repeat		reserved	none	0	0	1	0
0	20	1						
22	integer		type	integer	0	0	1	0
0	21	1						
23	real		type	real	0	0	1	0
0	22	1						
24	boolean		type	boolean	0	0	1	0
0	23	1						
25	char		type	char	0	0	1	0
0	24	1						
26	string		type	string	0	0	1	0
0	25	1						
27	then		reserved	none	0	0	1	0
0	26	1						
28	to		reserved	none	0	0	1	0
0	27	1						
29	type		reserved	none	0	0	1	0
0	28	1						
30	until		reserved	none	0	0	1	0
0	29	1						
31	var		reserved	none	0	0	1	0
0	30	1						
32	while		reserved	none	0	0	1	0
0	31	1						
33	true		constant	boolean	0	0	1	0
1	32	1	true					
34	false		constant	boolean	0	0	1	0
0	33	1	false					
35	readln		procedure	none	0	0	1	0
0	34	1						
36	write		procedure	none	0	0	1	0
0	35	1						
37	writeln		procedure	none	0	0	1	0
0	36	1						
38	definiteassignmentprogram			none	0	0	1	0
0	37	1						
39	x		variable	integer	0	0	1	0
3	38	1						
40	y		variable	integer	0	0	1	0
4	39	1						

btab

idx	kind	last	lpar	psze	vsze
0	program	40	0	0	2
1	compound	0	0	0	0

atab

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

----- Semantic Diagnostics -----

error: Variable is used before definite initialization: x

Pengujian ini memastikan bahwa program tidak melanjutkan proses ke tahap Intermediate Code Generation jika masih terdapat semantic error. Dengan demikian, Decorated AST dan Symbol Table tetap dapat ditampilkan, tetapi bagian Intermediate Code dan eksekusi interpreter tidak dijalankan.

Pengujian 7

Diberikan input sebagai berikut:

```
program RuntimeBounds;

var
  values: array[1..2] of integer;
  index: integer;

begin
  index := 3;
  values[index] := 1
end.
```

Dihasilkan output (build/runtime-error.txt) sebagai berikut:

[ERROR] ip 6: array index 3 outside 1..2

Pengujian ini memastikan bahwa interpreter melakukan validasi runtime terhadap akses array. Ketika indeks yang digunakan berada di luar batas array, interpreter menghasilkan runtime error dan menghentikan eksekusi program.

Pengujian 8

Diberikan input sebagai berikut:

```
program ArtifactRoundTrip;

type
  Pair == record
    left: integer;
    right: integer
  end;

var
  value: Pair;

begin
  value.left := 8;
  value.right := 9;
  writeln(value.left, value.right)
end.
```

Dihasilkan output sebagai berikut:

89

Pengujian ini memastikan bahwa Milestone 4 dapat dijalankan dari hasil tahap sebelumnya, yaitu Decorated AST dan Symbol Table. Program pertama-tama menghasilkan output dari source code, kemudian output tersebut dibaca kembali sebagai input untuk membangkitkan Intermediate Code dan menjalankan interpreter tanpa menduplikasi bagian output lama.

Kesimpulan dan Saran

Kesimpulan

Tahap Milestone 4 merupakan tahap akhir dari pipeline Arion Compiler, yaitu mengubah hasil front-end berupa Decorated AST dan Symbol Table menjadi Intermediate Code, kemudian menjalankannya menggunakan interpreter berbasis Stack Machine. Berdasarkan perancangan dan implementasi yang telah dilakukan, program berhasil membangkitkan Intermediate Code dari Decorated AST dan menjalankannya hingga menghasilkan output program yang sesuai.

Program juga berhasil mengimplementasikan mekanisme eksekusi berbasis Stack Machine dengan pemisahan activation frame dan operand stack. Mekanisme ini memungkinkan interpreter menangani penyimpanan variabel, operand sementara, parameter, return value, pemanggilan procedure/function, nested scope, serta rekursi secara lebih terstruktur. Selain itu, program juga mendukung fitur bahasa seperti operasi aritmatika dan boolean, control flow, input/output, array, record, composite assignment, parameter composite, dan function dengan return value composite.

Dari sisi keamanan eksekusi, interpreter telah dilengkapi dengan validasi runtime dan error handling. Validasi yang diterapkan mencakup stack overflow dan underflow, invalid jump, division by zero, bounds checking pada array, validasi tipe runtime, validasi call-return, serta perlindungan terhadap akses memori yang tidak valid. Hal ini membuat program tidak langsung crash ketika menemukan kesalahan saat runtime, melainkan dapat menghentikan eksekusi secara terkendali dengan pesan error yang informatif.

Berdasarkan pengujian yang dilakukan, program mampu menangani berbagai skenario, mulai dari assignment sederhana, procedure/function dan rekursi, nested scope, array dan record, input/output, semantic gating, runtime bounds error, hingga pembacaan kembali artifact Decorated AST. Dengan demikian, implementasi Milestone 4 telah memenuhi tujuan utama, yaitu menjadikan hasil analisis semantik dari milestone sebelumnya dapat diterjemahkan dan dieksekusi sebagai program Arion yang berjalan secara nyata.

Saran

Terlepas dari program yang telah berjalan dengan baik, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan selanjutnya.

1. **Kelompok ISL**

Program dapat dikembangkan lebih lanjut dengan menambahkan dokumentasi teknis yang lebih rinci pada bagian konvensi Intermediate Code, terutama untuk instruksi tambahan seperti `LDA`, `LDI`, `STI`, `BLD`, `BST`, `ADI`, dan `IXA`. Dokumentasi ini akan mempermudah proses debugging dan pengembangan lanjutan, terutama karena instruksi tersebut berperan penting dalam akses l-value, array, record, dan composite value.

2. **Pembuat Spesifikasi**

Spesifikasi milestone dapat dilengkapi dengan contoh yang lebih beragam, khususnya untuk kasus procedure/function, nested scope, array, record, parameter composite, return value composite, dan runtime error. Contoh-contoh tersebut akan membantu mengurangi ambiguitas implementasi antar kelompok, terutama pada bagian layout memori, konvensi call-return, serta instruksi tambahan di luar instruction set dasar.

3. **Pengembang Selanjutnya**

Pengembangan selanjutnya dapat difokuskan pada optimasi Intermediate Code dan peningkatan kualitas interpreter. Beberapa optimasi yang dapat dilakukan antara lain penghapusan instruksi yang tidak diperlukan, constant folding, penyederhanaan jump berantai, serta validasi tambahan untuk memastikan Intermediate Code yang dihasilkan lebih ringkas dan efisien. Selain itu, dukungan debugging seperti trace eksekusi, tampilan isi stack, dan tampilan perubahan frame juga dapat ditambahkan agar proses analisis program menjadi lebih mudah.

Lampiran

Link Github: <https://github.com/raulinn/ISL-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Bagian	Persentase Keseluruhan
13524044	Narendra Dharma Wistara M	Interpreter, Runtime Stack, Runtime Value, serta error handling pada tahap runtime seperti stack underflow, invalid jump, division by zero, dan runtime type error.	25%
13524060	Steven Tan	Translasi control flow dan call, meliputi <code>if</code> , <code>while</code> , <code>repeat</code> , <code>for</code> , <code>case</code> , procedure/function call, serta I/O seperti <code>write</code> , <code>writeln</code> , dan <code>readln</code> .	25%
13524090	Nashiruddin Akram	Intermediate Code Generator untuk ekspresi dan operasi, meliputi literal, identifier, variable, unary/binary expression, serta mapping operator ke instruksi OPR.	25%
13524100	Arghawisesa Dwinanda Arham	Struktur Intermediate Code dan code generation core, meliputi <code>Instruction</code> , <code>OpCode</code> , <code>OpCode</code> , <code>CodeGenContext</code> , assignment, deklarasi, address runtime, dan patching instruksi.	25%

Referensi

- [1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to automata theory, languages, and computation*, 3. ed. Boston San Francisco Munich: Pearson, 2007.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, & Tools*, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2007.
- [3] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.
- [4] Laboratorium Ilmu Rekayasa dan Komputasi, "Spesifikasi Milestone 4 - Tubes IF2224 TBFO", Institut Teknologi Bandung, Bandung, 2026. [Online]. Available: [Spesifikasi Milestone 4 - Tubes IF2224 TBFO](#)
- [5] Laboratorium Ilmu Rekayasa dan Komputasi, "Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi Intermediate Code", Institut Teknologi Bandung, Bandung, 2026. Available: [Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi Intermediate Code](#)